Coding & Solution

Date	**01 October 2026**
Team ID	SWTID-2026-8237
Project Name	Comic Craft - AI Comic Story Creator using Gemini Models
Maximum Marks	5 Marks

Field Details

Field	**Details**
Repository Link / URL	Not provided in the supplied project document
Programming Language(s)	Python, HTML, CSS, Jinja2
Framework(s) Used	Fast API, Unicorn, Jinja2
Key Features Implemented	AI comic outline generation; detailed narration and dialogue generation; AI comic illustration generation; panel layout construction; web-based comic preview; PDF export; form-based and JSON-based comic generation; image-generation test route; export success page.
Pending / Incomplete Features	The supplied project document identifies future enhancements such as user profiles, comic libraries, and multi-page story arcs. The current version focuses on single comic generation without user account management.
Setup / Run Instructions	Create and activate a Python virtual environment; install dependencies using requirements.txt or the documented pip packages; configure GEMINI_API_KEY and HF_API_KEY; run `unicorn app.main:app --reload`; open http://127.0.0.1:8000; use /docs for API testing.

Coding & Solution

This section evaluates the quality and completeness of the implemented Comic Craft solution based on the supplied project documentation. The solution uses a Fast API backend, HTML/CSS/Jinja2 frontend, Gemini Flash and Gemini Pro for text generation, and Stable Diffusion through Hugging Face Diffusers for comic illustrations.

The documented implementation separates the major operations into reusable modules: generate _outline(), generate _story(), generate _image(), build _comic _layout(), and save _pdf(). Fast API routes coordinate these functions for browser-based and JSON-based comic generation.

Instructions

Ensure the code is well-structured, commented, and follows best practices for the chosen technology stack.

The solution should address the core problem statement and implement the key functional requirements.

Include source files, configuration files, and setup instructions needed to run the application.

Attach or link to the code repository when a GitHub/GitLab/hosted repository is available.

Solution Summary

Comic Craft accepts a story prompt, main character name, setting, story tone, and art style through a web interface. Gemini Flash creates a structured five-panel outline, Gemini Pro expands it into narration and dialogue, and Stable Diffusion generates comic-style illustrations. The generated content is organized into a panel layout, displayed through a Jinja2-powered preview page, and exported as a multi-page PDF using FPDF.

Code Quality Checklist

S. No	Criteria	Status (Yes / No)	Evidence / Comments
1	Code is modular and organized into functions / classes	Yes	The project separates routes.py, gemini_flash.py, gemini_pro.py, image_generator.py, layout_builder.py and exporters.py. Core operations are implemented as distinct functions.
2	Meaningful variable and function names are used	Yes	Functions such as generate _outline(), generate _story(), generate _image(), build _comic _layout() and save _pdf() clearly describe their responsibilities.
3	Code includes comments / documentation where necessary	Partial	The supplied project documentation provides detailed module and workflow explanations, but the complete

			source-code comment coverage is not shown in the supplied material.
4	Error handling is implemented for critical operations	Yes	Fast API route handlers use HTTP Exception and try-except blocks for unexpected failures such as API errors or JSON decode errors.
5	The application runs without critical errors	Yes	The project documents local deployment using Unicorn and provides the application URL and API documentation endpoint for testing.
6	Code is committed to a version control repository	Not verified	The supplied project documentation mentions Git as a prerequisite, but no repository URL or commit evidence is provided.

Additional Notes / Comments:

The Comic Craft implementation demonstrates an end-to-end workflow from user prompt to downloadable comic. The documented architecture supports modular maintenance and future enhancement. The project documentation specifically identifies future possibilities including user profiles, comic libraries, and multi-page story arcs. Repository information was not supplied, so a repository link cannot be verified or added.